

NUMA Binding for Fusion Compiler-Class Physical Design Jobs

A detailed design and operations white paper for `numa-fc-bind`

Companion artifacts

Artifact	Path
Animated mechanism GIF	demo/numa_fc_bind.gif
Still frame	demo/numa_fc_bind_still.png
Working code	scripts/, src/, examples/
This PDF	docs/numa_fc_whitepaper.pdf

Scope: Align CPU affinity and DRAM allocation to one NUMA node for a single fat PD job when the working set fits that node’s free memory.

Non-scope: Invented speedups on single-socket hosts; changing PD algorithms or QoR.

Abstract

Physical-design tools such as Synopsys Fusion Compiler allocate multi-gigabyte (often multi-tens-of-GB) working sets and spawn many worker threads. On dual- and quad-socket Linux compute nodes, DRAM is partitioned by **NUMA**: each socket owns a set of cores and a set of memory controllers. A load that misses into the other socket’s memory crosses an interconnect (Intel UPI / AMD Infinity Fabric) and pays both **latency** and **bandwidth** tax under contention.

Because Linux commonly places newly written anonymous pages by **first-touch**, and because farm schedulers may migrate threads across sockets, a job can run with CPUs that look 100% busy in `top` while the hot heap sits on the remote node. Wall-clock suffers; misdiagnosis is common (“need more cores”).

The operational response—when peak RSS **fits** one node’s free RAM—is:

```
numactl --cpunodebind=N --membind=N <eda_command>
```

This white paper documents the failure mode with figures, the policy knobs, the safety model, demos of the `numa-fc-bind` toolkit (including intentional refusal on single-node hosts), the hardware-only measurement path, and an explicit claims policy. The repository GIF is a **mechanism** visualization, not a benchmark.

Animated GIF in the repository

Path: `demo/numa_fc_bind.gif`

Regenerate: `make gif`

Labeled on every frame: mechanism · not a benchmark

Table of contents

1. [Motivation and industry context](#)
 2. [Problem statement](#)
 3. [NUMA primer with figures](#)
 4. [First-touch and the unbound failure mode](#)
 5. [The binding policy](#)
 6. [Safety model](#)
 7. [Operations flow](#)
 8. [Implementation in numa-fc-bind](#)
 9. [Demos on this host](#)
 10. [Measurement methodology](#)
 11. [Mechanism GIF storyboard](#)
 12. [Relationship to tmpfs + rsync](#)
 13. [When it helps — and when it does not](#)
 14. [Claims policy](#)
 15. [Defending in review / interview](#)
 16. [Limitations and future work](#)
 17. [Conclusion](#)
 18. [Appendix A — command cookbook](#)
 19. [Appendix B — glossary](#)
-

1. Motivation and industry context

1.1 The multi-socket PD farm

A typical block-level or SoC PD host may be:

- 2 sockets (2S) or 4 sockets,
- tens of cores per socket,
- hundreds of GB of DRAM split across NUMA nodes,
- shared with other jobs, license servers, and NFS clients.

Fusion Compiler / Innovus / large STA and extraction engines:

- allocate large heaps (netlist, timing graph, routing / extraction DB),
- stream and chase pointers through those structures,
- run multi-threaded phases that look CPU-bound in the scheduler.

1.2 Misdiagnosis

Teams often respond with more threads, larger machines, or algorithmic tuning when the binding constraint is **memory locality**. Conversely, hard-binding a job that does not fit one node's free RAM produces OOM or reclaim storms that get blamed on "numactl is broken."

numa-fc-bind exists to make the correct middle path **operable, measurable on real multi-socket silicon**, and **honest on single-node CI/cloud hosts**.

2. Problem statement

Given a PD tool command on a multi-socket Linux host, place its threads and anonymous memory on the same NUMA node when safe, without changing the tool binary or functional results, and without hard-binding into out-of-memory.

Success criteria

1. Functional equivalence (same Tcl, same deliverables).
 2. On ≥ 2 -node hardware: measurable microbench contrast (local vs remote membind) and/or better stage wall time with healthier numastat.
 3. Wrapper refuses unsafe hard membind when MemFree is too low (unless explicitly overridden).
 4. No citeable "speedup" artifacts from single-node hosts.
-

3. NUMA primer with figures

NUMA = Non-Uniform Memory Access. Local DRAM (same node as the CPU) is cheaper than remote DRAM (other node).

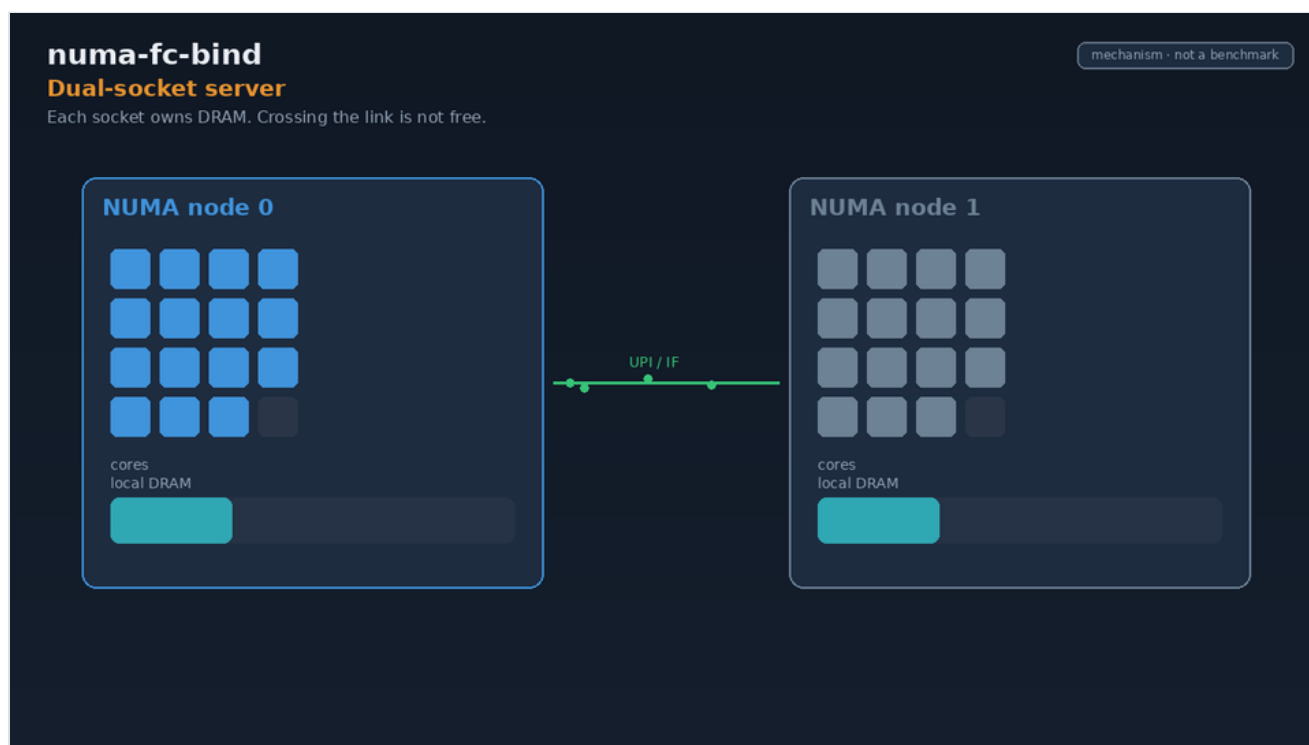


Figure: Dual-socket schematic from the mechanism GIF — each node owns cores and local DRAM; UPI/IF connects them.

Access	Typical cost shape
Local DRAM	baseline latency, full local controller bandwidth
Remote DRAM	higher latency, lower effective bandwidth under load, interconnect contention

Discovery on any Linux host:

```
./scripts/numa_report.sh  
# or  
numactl -H  
lscpu | grep NUMA
```

4. First-touch and the unbound failure mode

Linux often backs a newly written anonymous page on the **node of the CPU that first wrote it** (first-touch).

Canonical PD failure mode:

1. Master / parent thread on node 0 allocates and zeros a huge arena — **or** init lands on node 1.
2. Worker threads run hot on the other node.
3. Every miss walks the interconnect.
4. top shows busy CPUs; wall time does not match “CPU-bound” intuition.

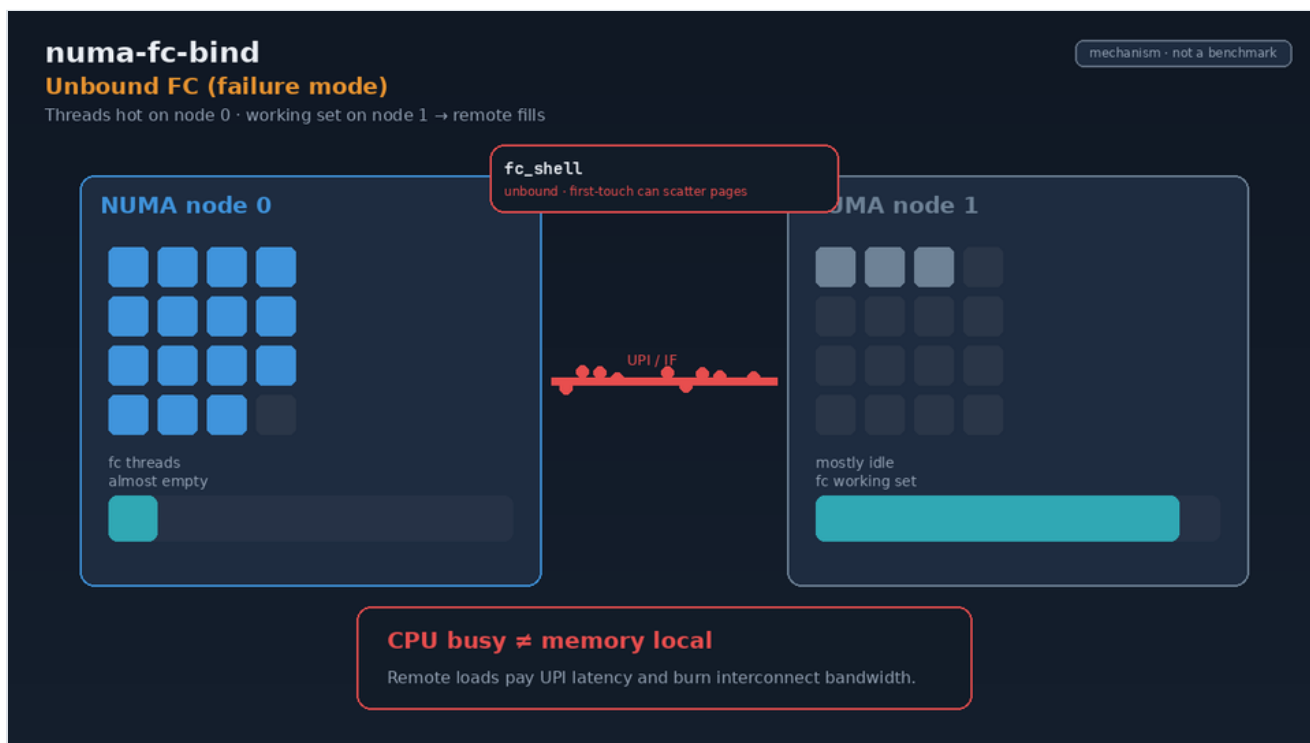


Figure: Unbound FC — threads hot on node 0, working set on node 1, UPI busy. Caption in-tool: “CPU busy ≠ memory local.”



Figure: Side-by-side schematic of remote fills vs local bind.

AutoNUMA / migrate-on-fault can move pages, but not reliably for short phases or huge stable working sets. Overnight production runs benefit from **explicit** policy.

5. The binding policy



Figure: The policy types out — `numactl --cpunodebind=0 --membind=0 fc_shell -f run.tcl`.

Flag	Meaning
<code>--cpunodebind=N</code>	Schedule only on node N CPUs
<code>--membind=N</code>	Allocate only from node N memory (hard)
<code>--preferred=N</code>	Prefer node N; spill elsewhere if needed (soft)
<code>--localalloc</code>	Prefer the touching CPU's node
<code>--interleave=...</code>	Stripe pages (sometimes for intentionally machine-sized heaps)

Primary recipe when the job fits node 0:

```
./scripts/run_with_numactl.sh 0 fc_shell -f run.tcl
```



Figure: Bound / local — CPU + pages on node 0; node 1 left alone; interconnect quiet.

6. Safety model

Hard membind on a node with insufficient free memory causes allocation failure or reclaim storms.

Guard	Default	Behavior
MIN_MEMFREE_GIB	8	Refuse hard membind if node MemFree is below this
FORCE=1	off	Explicit override (logged as WARNING)
POLICY=preferred	optional	Soft bind when RSS is uncertain
Single-node warning	always	Binding will not change DRAM locality

Rule of thumb: peak RSS × safety factor must fit the target node's free RAM. If it does not, do not hard-bind; resize the machine, split the job, or use --preferred.

7. Operations flow

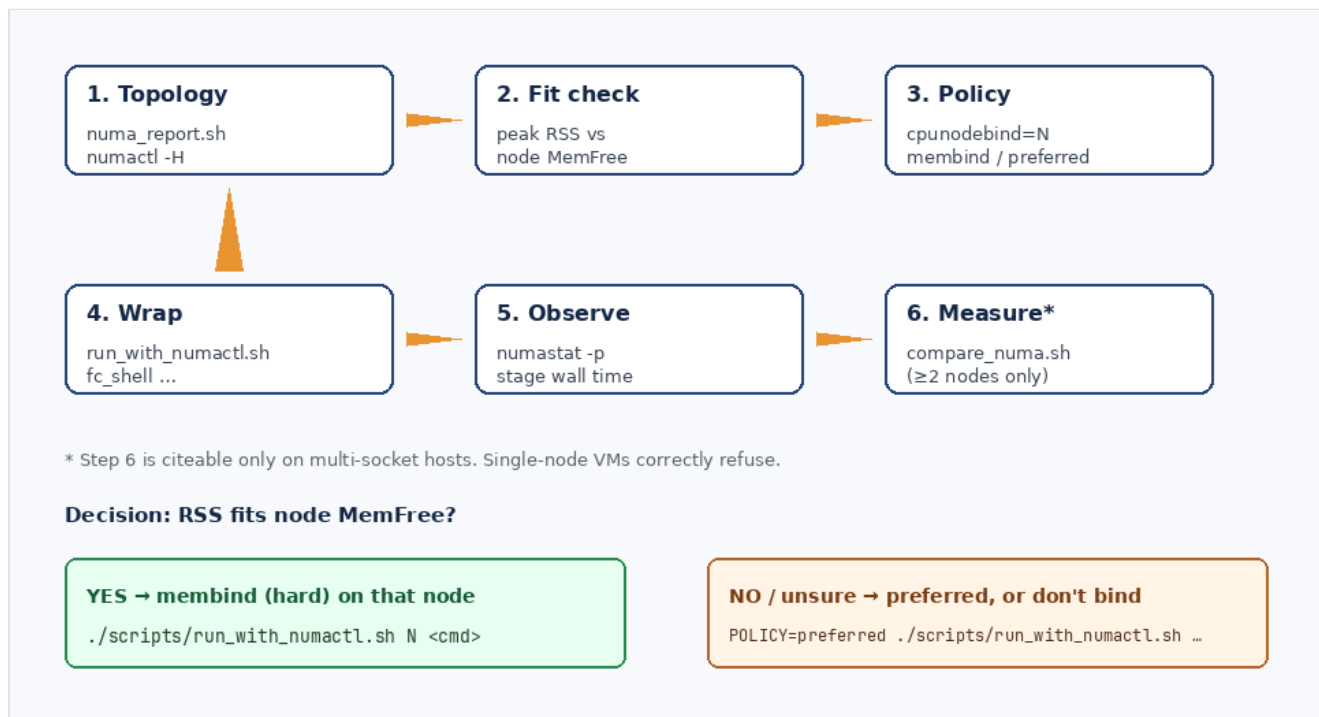


Figure: End-to-end ops flow and the MemFree decision gate.

1. **Topology** — `./scripts/numa_report.sh`
2. **Fit check** — peak RSS vs node MemFree
3. **Policy** — membind or preferred
4. **Wrap** — `./scripts/run_with_numactl.sh N <cmd>`
5. **Observe** — `numastat -p <pid>`, stage wall time
6. **Measure** — `./examples/compare_numa.sh` on ≥ 2 -node hosts only

8. Implementation in `numa-fc-bind`

Path	Role
<code>scripts/numa_report.sh</code>	Topology + MemFree + recommendation
<code>scripts/run_with_numactl.sh</code>	Production wrapper
<code>src/numa_mem_bench.cpp</code>	STREAM triad + pointer chase
<code>examples/compare_numa.sh</code>	Hardware remote vs local (median of trials)
<code>tools/numactl-root/usr/bin/</code>	Vendored numactl, numastat
<code>demo/gen_gif.py</code>	Mechanism GIF generator
<code>demo/numa_fc_bind.gif</code>	GIF checked into the repository
<code>docs/gen_figures.py</code>	Regenerates all white-paper figures + demo captures

9. Demos on this host

These captures were generated by running the toolkit on the documentation build host. They are **demos of operability**, not farm speedup claims.

9.1 Topology report

Demo — ./scripts/numa_report.sh (this host)

```
=====
NUMA report
=====
host: cursor date: 2026-08-22T01:42:42+00:00

---- lscpu ----
CPU(s):                                4
On-line CPU(s) list:                   0-3
Model name:                            Intel(R) Xeon(R) Processor
Thread(s) per core:                    1
Core(s) per socket:                    4
Socket(s):                             1
NUMA node(s):                          1
NUMA node0 CPU(s):                     0-3

---- numactl -H ----
available: 1 nodes (0)
node 0 cpus: 0 1 2 3
node 0 size: 16014 MB
node 0 free: 7902 MB
node distances:
```

Figure: Live `./scripts/numa_report.sh` output. This host is single-node — the script says so explicitly.

9.2 Microbench smoke

Demo — make smoke / numa_mem_bench (1-node smoke only)

```
$ ./build/numa_mem_bench --bytes 128M --threads 4

numa_mem_bench
  threads=4 bytes=134217720 chase_nodes=33554432
  affinity=0,1,2,3 numa_nodes=1
  STREAM triad: 107.823 GiB/s (higher better)
  pointer chase: 53.5081 ns/hop (lower better)
```

Figure: `numa_mem_bench` smoke run. Useful to prove the binary works; not a NUMA remote-vs-local result.

9.3 Compare correctly refuses on 1-node

Demo — compare_numa.sh on 1-node host (correct refusal)

```
$ ./examples/compare_numa.sh
# exit=2
ERROR: need ≥2 NUMA nodes (host has 1).
Refusing to write a fake remote-vs-local SUMMARY.
Useful on this host: ./scripts/numa_report.sh | make smoke | make gif
```

Figure: `./examples/compare_numa.sh` exits 2 and refuses to invent a remote-vs-local SUMMARY. This is required honesty.

9.4 Production wrapper

Demo — run_with_numactl.sh 0 /bin/true (FORCE=1 for low MemFree)

```
$ FORCE=1 ./scripts/run_with_numactl.sh 0 /bin/true

[run_with_numactl] host=cursor date=2026-08-22T01:42:43+00:00
[run_with_numactl] nodes=1 target=0 MemTotal≈15.64G MemFree≈7.65G
[run_with_numactl] policy=membind cmd: /bin/true
[numa] available: 1 nodes (0)
[numa] node 0 cpus: 0 1 2 3
[numa] node 0 size: 16014 MB
[numa] node 0 free: 7830 MB
[numa] node distances:
[numa] node 0
[numa] 0: 10
WARNING: FORCE=1 overriding MemFree guard
[run_with_numactl] exec: numactl --cpunodebind=0 --membind=0 /bin/true
[run_with_numactl] later: /workspace/numa-fc-bind/tools/numactl-root/usr/bin/numastat -p <pid>
WARNING: only 1 NUMA node(s) — binding will not change DRAM locality.
```

Figure: `FORCE=1 ./scripts/run_with_numactl.sh 0 /bin/true` — logs topology, MemFree, policy, then execs `numactl`.

10. Measurement methodology

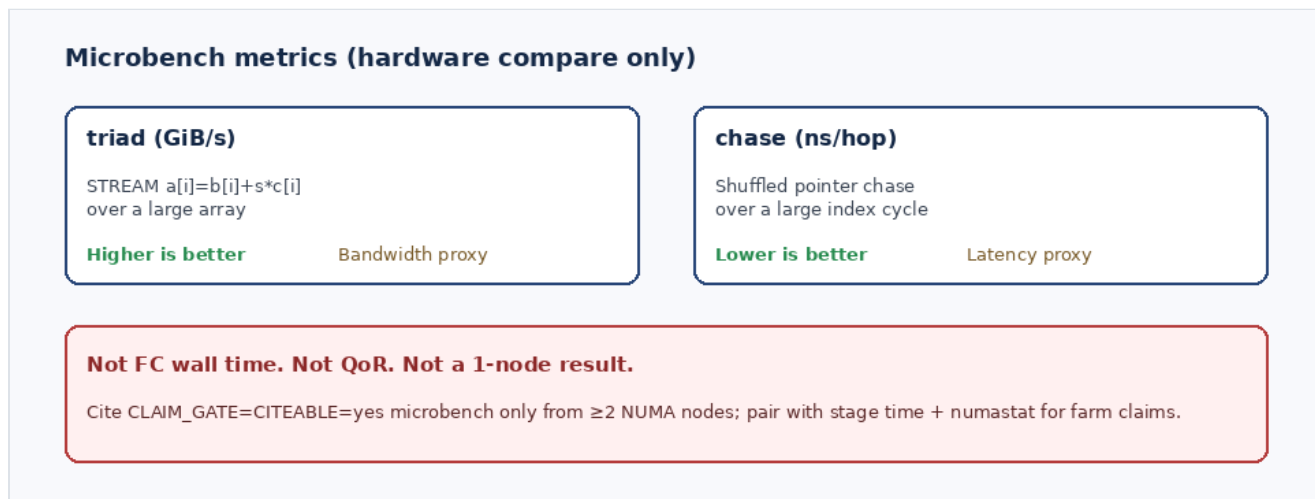


Figure: What triad and chase mean — and what they are not.

10.1 Hardware compare (citeable as microbench)

On a host with ≥ 2 NUMA nodes:

```
TRIALS=5 BYTES=1G THREADS=$(nproc) ./examples/compare_numa.sh
cat examples/compare_results/SUMMARY.txt
cat examples/compare_results/CLAIM_GATE.txt # must read CITEABLE=yes
```

Leg	Command
unbound	<code>./build/numa_mem_bench ...</code>
REMOTE	<code>numactl --cpunodebind=0 --membind=1 ./build/numa_mem_bench ...</code>
LOCAL	<code>numactl --cpunodebind=0 --membind=0 ./build/numa_mem_bench ...</code>

Aggregate: **median** of TRIALS, with min/max spreads printed so noise is visible.

10.2 Strongest farm claim

Microbench alone is insufficient for a customer or design-review claim about Fusion Compiler. Prefer:

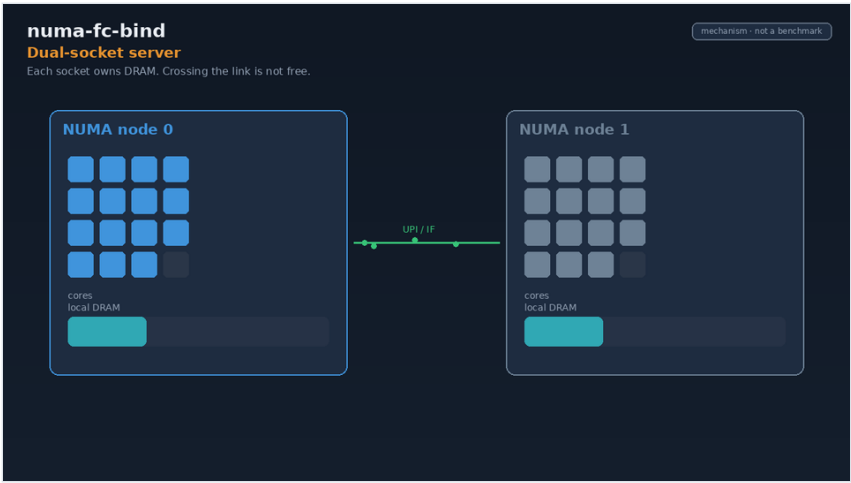
1. Stage wall time before/after policy (same design, same machine class),
2. `numastat -p <pid>` showing pages local to the bound node,
3. Optional hardware microbench SUMMARY attached as supporting evidence.

10.3 What this CI/cloud host cannot provide

Many documentation and agent VMs have **one** NUMA node. There is **no hardware remote-vs-local table** to publish from such a host. Absence of a speedup table in this paper is intentional and correct.

11. Mechanism GIF storyboard

The animated GIF is checked into the repo at `demo/numa_fc_bind.gif`. Every frame is badged **mechanism · not a benchmark**. Regenerate with `make gif`.

Scene	Figure	Message
Intro		Dual-socket; local DRAM per node
Unbound		Remote fills; CPU busy ≠ memory local
Policy		Type the numactl line

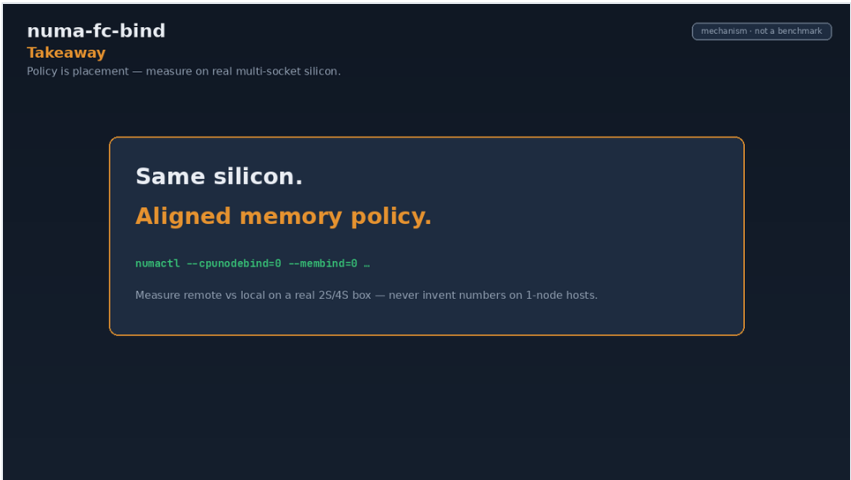
Scene	Figure	Message
Bound		Same node for cores + pages
Takeaway		Measure on real 2S/4S silicon



Figure: Still frame (*demo/numa_fc_bind_still.png*) — bound / local state.

12. Relationship to tmpfs + rsync

Concern	Lever	Project
NFS / disk latency, tiny-file storms	tmpfs hot tier + rsync durability	PD Job Acceleration
Remote DRAM / UPI contention	numactl CPU + mem bind	numa-fc-bind

A job can be I/O-bound in one phase and NUMA-bound in another. Profile each; apply the matching lever. They compose: wrap FC with `run_with_numactl.sh` and keep fat logs off NFS scratch when appropriate.

13. When it helps — and when it does not

Situation	Expectation
2S/4S, one FC, RSS fits node free RAM	Primary target
1 NUMA node	No DRAM-locality win from membind
RSS > node free RAM + hard membind	Harm (OOM / reclaim)
Bottleneck is NFS / disk	Fix I/O tiering first
Many small jobs	Prefer one heavy job per node
Intentionally machine-spanning heap	

Situation	Expectation
	Consider interleave / preferred, not hard single-node membind

14. Claims policy

Allowed	Not allowed
Hardware compare on ≥ 2 nodes with CLAIM_GATE=CITEABLE=yes, labeled as microbench	"X $\times$ faster" from a 1-node / cloud VM
Real FC stage wall time + numastat	Treating the GIF as measured performance
Explaining the recipe + safety rules	Hard membind into a node that cannot hold peak RSS
Publishing this paper's demos as operability evidence	Publishing refused compare output as a speedup

15. Defending in review / interview

1. **Topology** — `show numactl -H / numa_report.sh` (multi-node).
2. **Hypothesis** — remote fills / first-touch scatter.
3. **Measurement** — hardware microbench local vs remote **and/or** stage wall time + numastat.
4. **Safety** — MemFree vs peak RSS; refuse unsafe membind.
5. **Non-claims** — not QoR; not a substitute for I/O tiering; not a 1-node miracle.

One-liner: Same silicon, aligned memory policy — pin FC CPUs and pages to one NUMA node when the working set fits.

numa-fc-bind

mechanism · not a benchmark

Takeaway

Policy is placement — measure on real multi-socket silicon.

Same silicon.

Aligned memory policy.

```
numactl --cpunodebind=0 --membind=0 ..
```

Measure remote vs local on a real 2S/4S box — never invent numbers on 1-node hosts.

16. Limitations and future work

- No licensed FC binary in CI — wrapper + microbench only.
- Microbench $\neq$ full tool wall time.
- Future: cgroup/cpuset farm integration; automatic RSS-vs-MemFree advisor; perf c2c recipes; hugepage interaction notes.

17. Conclusion

NUMA binding is memory **placement**, not a new PD algorithm. It is useful on multi-socket farms when the job fits one node, dangerous when it does not, and unmeasurable as remote-vs-local on single-node hosts. numa-fc-bind ships working code, a detailed README, this illustrated white paper, and a repository GIF that explains the mechanism without fabricating numbers.

Appendix A — command cookbook

```
cd numa-fc-bind

# Topology
./scripts/numa_report.sh

# Build + smoke
make -j && make smoke
```

```
# Regenerate GIF + figures + PDF
make gif
python3 docs/gen_figures.py
make pdf

# Production wrap
./scripts/run_with_numactl.sh 0 fc_shell -f run.tcl
POLICY=preferred ./scripts/run_with_numactl.sh 0 fc_shell -f run.tcl

# Hardware compare (≥2 NUMA nodes)
TRIALS=5 BYTES=1G ./examples/compare_numa.sh
```

Appendix B — glossary

Term	Meaning
NUMA	Non-Uniform Memory Access
UPI / IF	Intel Ultra Path Interconnect / AMD Infinity Fabric
first-touch	Page placed on the node of the CPU that first writes it
membind	Hard memory-node binding
cpunodebind	Restrict threads to a node's CPUs
triad	STREAM triad bandwidth (GiB/s)
chase	Pointer-chase latency (ns/hop)
CLAIM_GATE	File stating whether a compare run is citeable